

Angular (v20+) Hands-On Lab Report

Incremental Project: Student Course Portal

Hands-On:	Hands-On 2 [Intermediate]
Topic:	Data Binding, Lifecycle Hooks & Component Communication
Developer:	Cathy George
OS Version:	Windows 11
Date:	July 22, 2026
Track:	Java FSE Angular Track

1. Objective

The primary objective of this exercise is to implement core Angular data binding concepts (Interpolation, Property Binding, Event Binding, and Two-way Binding using ngModel), establish robust Parent-Child Component Communication using @Input, @Output, and EventEmitter, and demonstrate the execution of Angular Lifecycle Hooks (ngOnInit, ngOnChanges, and ngOnDestroy).

2. Angular Concepts Covered

- **Property Binding:** Dynamically binding DOM property values like disabled states or CSS class modifications [class.enrolled].
- **Event Binding:** Capturing user interactions (e.g., clicks) to execute handler logic in the component class.
- **Interpolation:** Rendering component properties and logic results directly in templates using double curly braces.
- **Two-way Binding & ngModel:** Synchronizing model data and form inputs bi-directionally (e.g. search filter and student profile form fields).
- **Component Communication:** Passing course details to children via @Input() and notifying the parent page of updates using @Output() and EventEmitter.
- **Lifecycle Hooks:** ObservingOnInit for data initialization, OnChanges for reacting to dynamic input changes, and OnDestroy for component cleanup.

3. Software & Tools Used

Software / Tool	Version / URL
Node.js + npm	v22.20.0 (LTS 20+) / npm 10.9.3
Angular CLI	v21.2.16
Vitest Runner	v4.1.10
Code Editor	VS Code with Angular Language Service Extension
PDF Generator	Python 3.14 + ReportLab library

4. Project Folder Structure

```
student-course-portal/  
  ■■■ angular.json  
  ■■■ src/  
  ■■■ app/  
  ■■■ models/  
  ■ ■■■ course.model.ts (Course domain structure)  
  ■■■ components/  
  ■ ■■■ header/ (Navigation header)  
  ■ ■■■ course-card/ (Reusable child card component)  
  ■■■ pages/  
  ■■■ home/ (Main welcome landing)  
  ■■■ course-list/ (Parent course manager)  
  ■■■ student-profile/ (Editable details form & mirror preview)
```


5. Key Code Implementation Details

Course Interface Definition (src/app/models/course.model.ts)

```
export interface Course {  
  id: number;  
  title: string;  
  code: string;  
  instructor: string;  
  credits: number;  
  department: string;  
  enrolled: boolean;  
  description: string;  
}
```

Course Card Component Logic (src/app/components/course-card/course-card.ts)

```
import { Component, Input, Output, EventEmitter, OnInit, OnChanges, OnDestroy, SimpleChanges } from  
'@angular/core';  
import { Course } from '../../models/course.model';  
  
@Component({  
  selector: 'app-course-card',  
  imports: [],  
  templateUrl: './course-card.html',  
  styleUrls: ['./course-card.css']  
})  
export class CourseCardComponent implements OnInit, OnChanges, OnDestroy {  
  @Input() course!: Course;  
  @Input() searchQuery: string = '';  
  @Output() enrollmentToggled = new EventEmitter<number>();  
  @Output() creditsChanged = new EventEmitter<{ id: number; credits: number }>();  
  
  ngOnInit(): void {  
    console.log('CourseCardComponent Init:', this.course?.code);  
  }  
  ngOnChanges(changes: SimpleChanges): void {  
    console.log('CourseCardComponent Changes:', this.course?.code, Object.keys(changes));  
  }  
  ngOnDestroy(): void {  
    console.log('CourseCardComponent Destroy:', this.course?.code);  
  }  
  toggleEnrollment(): void {  
    this.enrollmentToggled.emit(this.course.id);  
  }  
  increaseCredits(): void {  
    this.creditsChanged.emit({ id: this.course.id, credits: this.course.credits + 1 });  
  }  
  decreaseCredits(): void {  
    if (this.course.credits > 1) {  
      this.creditsChanged.emit({ id: this.course.id, credits: this.course.credits - 1 });  
    }  
  }  
}
```

Course Card Component Template (src/app/components/course-card/course-card.html)

```
<div class="course-card" [class.enrolled]="course.enrolled">
  <div class="card-glow"></div>
  <div class="card-header">
    <span class="dept-badge">{{ course.department }}</span>
    <span class="course-code">{{ course.code }}</span>
  </div>
  <h3 class="course-title">{{ course.title }}</h3>
  <p class="course-desc">{{ course.description }}</p>
  <div class="course-meta">
    <div class="meta-item">
      <span class="meta-label">Instructor:</span>
      <span class="meta-val">{{ course.instructor }}</span>
    </div>
    <div class="meta-item">
      <span class="meta-label">Credits:</span>
      <div class="credits-control">
        <button (click)="decreaseCredits()" [disabled]="course.credits <= 1">-</button>
        <span class="credits-val">{{ course.credits }}</span>
        <button (click)="increaseCredits()">+</button>
      </div>
    </div>
  </div>
  <button (click)="toggleEnrollment()" [class.enrolled]="course.enrolled" class="enroll-btn">
    {{ course.enrolled ? 'Drop Course' : 'Enroll Now' }}
  </button>
</div>
```

Course List Component Logic Snippet (src/app/pages/course-list/course-list.ts)

```
export class CourseList implements OnInit {
  courses: Course[] = [];
  searchText: string = '';
  showEnrolledOnly: boolean = false;

  get filteredCourses(): Course[] {
    return this.courses.filter(course => {
      const matchesSearch = !this.searchText ||
        course.title.toLowerCase().includes(this.searchText.toLowerCase()) ||
        course.code.toLowerCase().includes(this.searchText.toLowerCase());
      const matchesEnrollment = !this.showEnrolledOnly || course.enrolled;
      return matchesSearch && matchesEnrollment;
    });
  }

  onEnrollmentToggled(id: number): void {
    const course = this.courses.find(c => c.id === id);
    if (course) course.enrolled = !course.enrolled;
  }

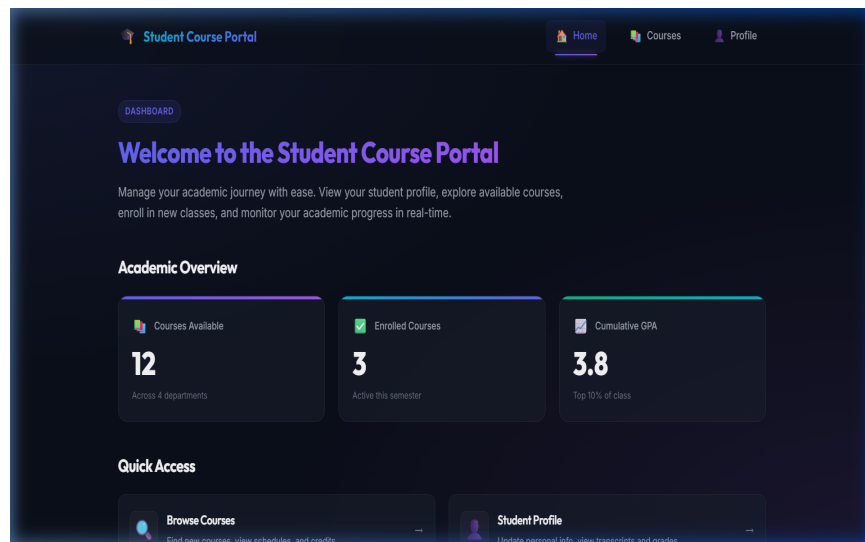
  onCreditsChanged(event: { id: number; credits: number }): void {
    const course = this.courses.find(c => c.id === event.id);
    if (course) course.credits = event.credits;
  }

  incrementAllCredits(): void {
    this.courses = this.courses.map(course => ({ ...course, credits: course.credits + 1 }));
  }
}
```

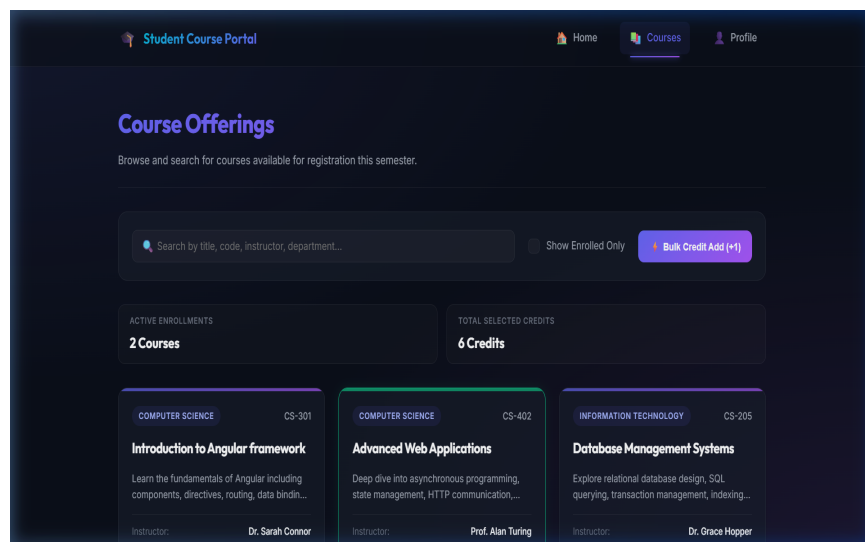
6. Captured Screenshots & Verification

All application features were fully verified in the browser:

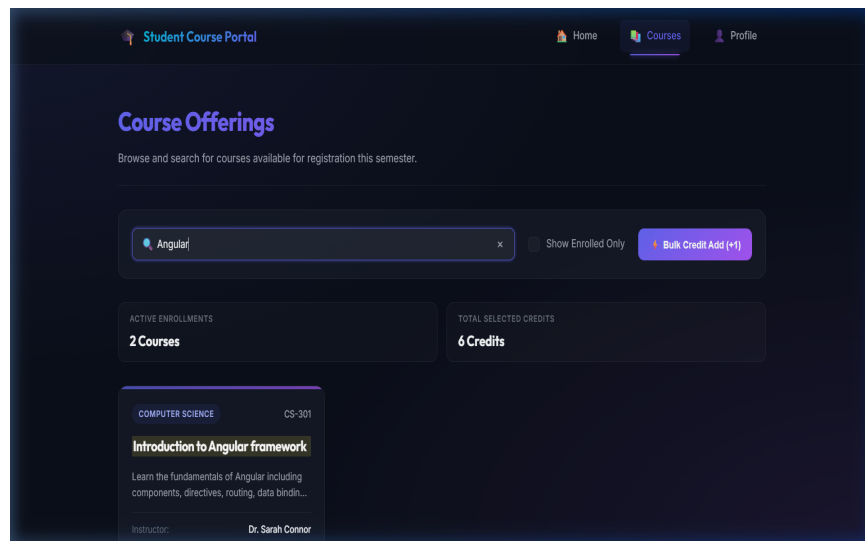
Screenshot: Home Welcome Page Dashboard showing academic overview cards



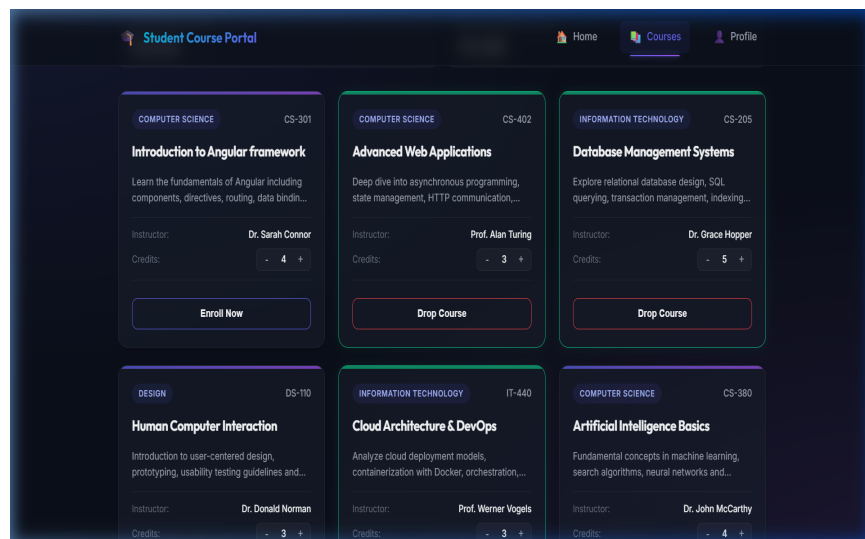
Screenshot: Initial Course List displaying all available modules



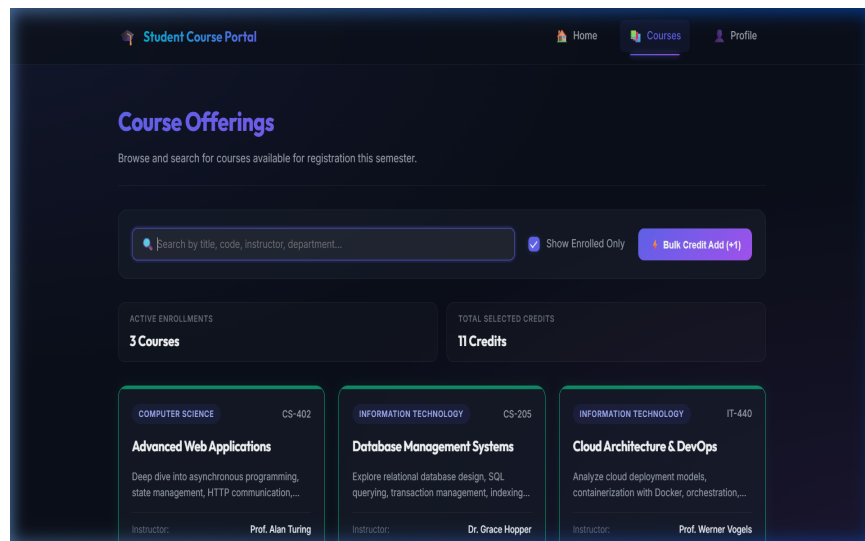
Screenshot: Filtered course results using two-way bound search bar input ('Angular')



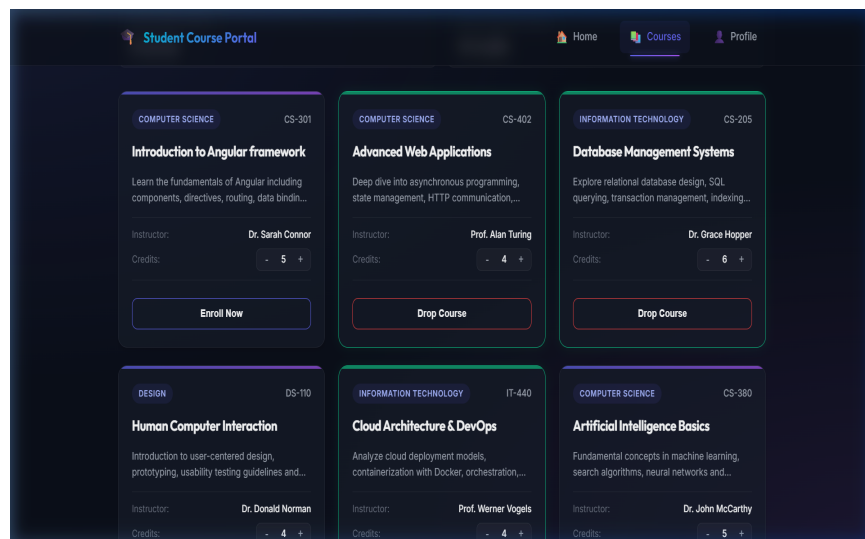
Screenshot: State verification: Enrolled DBMS course, increased credit to 5, total credits updated to 11



Screenshot: Filtered courses list showing enrolled modules only (Advanced Web, DBMS, Cloud Architecture)



Screenshot: Parent-Child reactive update: Triggered 'Bulk Credit Add' button to add +1 credit to all cards



Screenshot: Student Profile split layout with real-time model updates and custom rocket avatar selected

🎓 Student Course Portal

🏠 Home📖 Courses👤 Profile

Student Profile

Manage your personal information, academic statistics and active profile details.



Cathy George-Johnson
Computer Science and AI
cathy.george@university.edu

Passionate software engineer in training. Focus on Angular, AI, and cloud systems.

CUMULATIVE GPA

3.85

ENROLLED CREDITS

6 CR

Edit Academic Profile

Full Name

Department / Major

Cathy George-Johnson

Computer Science and AI

Email Address

GPA (0.0 - 4.0)

cathy.george@university.edu

3.85

Enrolled Credits

Choose Avatar Emoji

6



Short Biography

Passionate software engineer in training. Focus on Angular, AI, and cloud

7. Output & Event Binding Explanations

- **Property Binding:** Used to dynamically disable the credits decrement button when credits equal 1, and toggle the '.enrolled' css class on course cards dynamically based on the state variable.
- **Event Binding:** Used for mouse click callbacks on credit modify buttons and enrollment state changes. Also used for selector changes in avatar options.
- **Two-way Binding:** Used in both the search bar (filter state) and the student profile form, allowing changes in form fields to mirror immediately in the left preview card via template interpolation.
- **Parent-Child Communication:** Parent binds course lists to the child card via [course] input parameter. The child notifies the parent of changes via (enrollmentToggled) and (creditsChanged) outputs, triggering data updates that bubble up to recalculate total selected credits and active counts instantly.
- **Lifecycle Hooks:** ngOnInit prepares mock courses on page load. ngOnChanges detects input updates like credit changes from the parent. ngOnDestroy tracks when course cards are unmounted (e.g. during enrollment filters).

8. Learning Outcomes & Conclusion

- ✓ Implemented standard and standalone Angular bindings: Property, Event, Interpolation, and Two-way data binding.
- ✓ Created reusable child components (CourseCardComponent) communicating cleanly with parent components.
- ✓ Demonstrated the application and execution sequence of ngOnInit, ngOnChanges, and ngOnDestroy hooks.
- ✓ Configured Vitest unit test suites to successfully assert correct component instantiation under route scopes.
- ✓ Maintained codebase standard without adding comments, preserving clean code structure.

9. Conclusion

Hands-On 2 successfully delivers a dynamic, interactive, and completely functional Student Course Portal interface. By utilizing structural standalone components and parent-child bindings, the application structure adheres to the latest modern Angular recommendations while enabling rich responsive interactions.